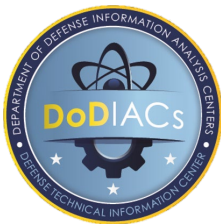


REPORT DOCUMENTATION PAGE			Form Approved OMB No. 0704-0188	
Public reporting burden for this collection of information is estimated to average 1 hour per response, including the time for reviewing instructions, searching existing data sources, gathering and maintaining the data needed, and completing and reviewing this collection of information. Send comments regarding this burden estimate or any other aspect of this collection of information, including suggestions for reducing this burden to Department of Defense, Washington Headquarters Services, Directorate for Information Operations and Reports (0704-0188), 1215 Jefferson Davis Highway, Suite 1204, Arlington, VA 22202-4302. Respondents should be aware that notwithstanding any other provision of law, no person shall be subject to any penalty for failing to comply with a collection of information if it does not display a currently valid OMB control number. PLEASE DO NOT RETURN YOUR FORM TO THE ABOVE ADDRESS.				
1. REPORT DATE (DD-MM-YYYY) 05-04-2025		2. REPORT TYPE Technical Report		3. DATES COVERED (From - To)
4. TITLE AND SUBTITLE Assembling a Deep-History Binary Corpus		5a. CONTRACT NUMBER FA8075-18-D-0004 (IAC MAC)		
		5b. GRANT NUMBER		
		5c. PROGRAM ELEMENT NUMBER		
6. AUTHOR(S) Ashok, Naveen; Raff, Edward; Holt, James; Micinski, Kristopher		5d. PROJECT NUMBER FA807523F0031		
		5e. TASK NUMBER 22-2393		
		5f. WORK UNIT NUMBER 3.7-8-1		
7. PERFORMING ORGANIZATION NAME(S) AND ADDRESS(ES) IAC MAC Booz Allen Hamilton 8283 Greensboro Drive McLean, VA 22102		8. PERFORMING ORGANIZATION REPORT NUMBER		
9. SPONSORING / MONITORING AGENCY NAME(S) AND ADDRESS(ES) Laboratory for Physical Sciences 5520 Research Park Drive, Suite 450 Catonsville, MD 21228		10. SPONSOR/MONITOR'S ACRONYM(S)		
		11. SPONSOR/MONITOR'S REPORT NUMBER(S)		
12. DISTRIBUTION / AVAILABILITY STATEMENT Distribution Statement A: Approved for public release: distribution is unlimited.				
13. SUPPLEMENTARY NOTES This deliverable was created by Booz Allen Hamilton, 8283 Greensboro Drive, McLean, VA 22102.				
14. ABSTRACT One area of the client's Advanced Computing Research mission is to drive innovations in computing platforms and foundational applications that run on them. Large Language Models (LLMs) have demonstrated tremendous impact in multiple areas and are finding broader impact in all code-related activities, to include reverse-engineering, code translation, and system design. Extracting information from binary executables via LLMs has potential to be of great benefit for many of these tasks but is hampered by lack of good binary data for the LLMs to train on. Some characteristics of a good data repository include volume, specific families (e.g. Windows Portable Executable (PE)), labels and metadata, repeatability, variety of compilations, and inclusion of long-term major software development efforts with deep development history. Previous work for the client was initial development of ASSEMBLAGE, a distributed system for automatically building binary corpuses from crawled repositories in a robust, reproducible, and customizable way. While effective, ASSEMBLAGE creates the data sets exclusively from Github content which would not include critical long-term developed software or capture development history. This research work addresses this gap by using human expertise in concert with ASSEMBLAGE to semi-automatically build the desired datasets. This report describes the deep customization of the build toolchain and the resulting binary corpora with "deep histories", developed via multi-year code repositories and comparing different major releases of open-source software. This set of 251 projects, containing 2,668,776 functions, provides a rich and high-quality corpus of input data. The utility of the new data set is demonstrated by performing three evaluations studying binary similarity, nearest neighbor search, and code evolution factors. Government and industry scientists seeking to use LLMs for code-related efforts will benefit from this information.				
15. SUBJECT TERMS IAC MAC, hpc, high performance computing, assemblage, code similarity, binary similarity, longitudinal data, reverse engineering, llm, large language model, kernel density, claude				
16. SECURITY CLASSIFICATION OF:			17. LIMITATION OF ABSTRACT Unlimited	18. NUMBER OF PAGES 13
a. REPORT UNCLASSIFIED	b. ABSTRACT UNCLASSIFIED	c. THIS PAGE UNCLASSIFIED		
				19a. NAME OF RESPONSIBLE PERSON Teresa Sorensen
				19b. TELEPHONE NUMBER (include area code) 703-984-7010



Assembling a Deep-History Binary Corpus

FA8075-18-D-0004, TO FA807523F0031, RMS 22-2393

5 April 2025

Distribution Statement A.

Approved for public release: distribution is unlimited.

Assembling a Deep-History Binary Corpus

Naveen Ashok
Syracuse University
nashok@syr.edu

James Holt
Laboratory for Physical Sciences
holt@lps.umd.edu

Edward Raff
Booz Allen Hamilton
Raff_Edward@bah.com

Kristopher Micinski
Syracuse University
kkmicins@syr.edu

ABSTRACT

Applying Knowledge Discovery (KD) and Information Retrieval (IR) methods to source code has become increasingly popular, especially as Large Language Models (LLMs) have provided a simple and easy method of obtaining code embeddings and querying for information about code. However, reverse engineering tasks have been highly neglected in this effort in part due to poor data availability. In this work, we produce a dataset of binary corpora with “deep histories”, via multi-year code repositories and comparing different major releases of open-source software. This set of 251 projects, containing 2,668,776 functions, provides a rich and high-quality corpus of input data—exclusively comprised of relatively large projects with deep histories. We build on ASSEMBLAGE, a binary corpus archival system, to reproducibly build these projects. To demonstrate the relevance of our deep history corpus, we perform three evaluations studying binary similarity, nearest neighbor search, and the relationship between code evolution factors and binary similarity.

CCS CONCEPTS

• Information systems → Data extraction and integration.

KEYWORDS

Code Similarity, Binary Similarity, Longitudinal Data, Reverse Engineering, Windows PE

1 INTRODUCTION

The emergence of Large Language Models (LLMs) is revolutionizing the way we develop, reason about, and understand code. LLMs [34] use unsupervised learning to perform pre-training over large input corpora (including source code), often followed by fine-tuning over more limited application-specific data. However, there is still a

significant gap in applying State-of-the-Art (SOTA) LLMs [9, 32, 36] to binary code for reverse engineering purposes, primarily due to the scarcity of comprehensive binary datasets with historical context [22, 31]. As a solution, there have been several significant efforts in providing publicly available training binary input data, with the largest available corpora including hundreds of thousands to millions of *Linux* binaries [5, 19, 22, 39]. Unfortunately, very little publicly available prior work included a meaningful number of Windows PE binaries [4, 37]—which are particularly important in studying malware and similar security-relevant tasks. Recently, our work ASSEMBLAGE [22] introduced the most extensive publicly-available corpus of Windows PE binaries (1.5M), assembled by automatically crawling GitHub and attempting to build binaries in a diverse array of configurations.

In this paper, we present a novel extension of ASSEMBLAGE, which targets large, complex projects with deep version histories. In our previous work on ASSEMBLAGE, there was a bias towards small software repositories due to the restriction for automatically compiling the code. Larger projects that better reflect complex code require custom installation and dependencies that are not trivially deducible in an automated way via MSBuild [25]. ASSEMBLAGE will trivially miss any projects not hosted on Github as an additional bias. Additionally, ASSEMBLAGE was previously designed to scan GitHub at a single commit in time, providing no insight into the history and evolution of software.

Our work addresses all of these gaps by introducing a novel dataset of binary corpora with “deep histories:” large open-source software projects with development histories that span *years*. We first extended ASSEMBLAGE to enable custom build recipes which enable configuring various aspects of the build toolchain to enable reproducible builds that diversify the project across multiple versions. Our infrastructure robustly supports various existing package managers (e.g., *vcpkg* [26]) while also supporting low-level customization (to install new frameworks, change environment variables, compiler flags, etc...). Using these extensions, we expended several human weeks (assisted by an LLM where we manually edited/audited output) to semi-automatically write scripts to build 4–6 versions of 90 projects. We also included 161 packages from Conan [17], a modern Windows package manager that enabled us to build various versions.

In sum, our contributions are as follows:

- We extend ASSEMBLAGE to enable deep customization of the build toolchain, enabling us to build large, complex open-source repositories which were previously beyond the reach of ASSEMBLAGE’s fully-automatic machinery.

- We present a “deep-history” binary corpus comprising 251 total packages; there are 135 packages with 6 or more versions, 51 with 5 versions and 65 with 4 versions. The dataset contains 2,565 .exe files, 3,026 .dll files, and 5,472 .lib binaries, along with fine-grained metadata and .pdb files (crucial for retaining ground-truth). We are imminently planning to release our data on HuggingFace.
- We present an evaluation of our deep history dataset, studying its properties and illustrating the potential for reasoning about the evolution of binaries across time.

2 RELATED WORK

As we mentioned previously, our deep history dataset is an extension of ASSEMBLAGE [22]. Assemblage addressed the critical need for high-quality corpora of Windows PE binaries by developing a distributed system capable of crawling, configuring, and building open-source repositories. In sum, ASSEMBLAGE includes 890k Windows PE and 428k Linux ELF binaries across 29 build configurations. While Assemblage provided the foundational infrastructure, our work extends its capabilities to capture multiple versions of projects, directly addressing the need for historical context in binary analysis. We now mention several threads of related work.

Binary Similarity and Existing Corpora. Recent advances in binary similarity detection have demonstrated the growing importance of comprehensive binary datasets. PalmTree [21] was one of the earlier deep learning methods used to embed assembly instructions. BinProv [15] leverages BERT-based embedding models to learn contextual semantics directly from binary code for effective binary code similarity detection. EMBERSim enhanced the EMBER dataset with similarity information and malware class tags [1, 6]. SOREL-20M is a large dataset that pertains to malware samples [14]. They provide various pre-extracted features useful to train models for detecting specific versions of malware. STRIDE is a simple NGram based technique of predicting variable names from decompiled output [13]. MOTIF provides a large malware reference dataset, particularly including a large number of Windows malware [18]. BinBench [5] introduced a multi-task benchmark for evaluating machine learning models on assembly functions. Recent models including VulHawk’s [23] entropy-based binary code search and GTrans’s [38] graph transformer-based approach demonstrate the field’s movement toward deep-learning-based analysis techniques. These techniques, particularly GTrans’s focus on obfuscation resilience, highlight the importance of diverse, well-documented binary datasets that span multiple versions and configurations. The importance of our deep-history approach is underscored by the findings presented in REFUSE, of Saul et. al [30]. REFUSE revealed that—by training convolutional models on a diverse set of binaries—one can achieve state-of-the-art performance in function similarity for Windows PE binaries

LLMs for Decompilation/Reverse Engineering. Our dataset’s historical depth complements REFUSE’s findings by providing the temporal dimension necessary to study how binary signatures evolve while maintaining functional equivalence. Recent advances in applying Large Language Models (LLMs) to binary analysis,

Table 1: Number of projects per category

Category	Projects
Graphics and Image manipulation	52
Parsing	27
System tools	25
Networking	23
Compression	21
Databases	19
File formats	19
Data structures	18
ML and math	16
API clients	12
OS APIs	10
Formatting	10
Cryptographic	8
Hardware	5

as demonstrated by LLM4Decompile [32] and ReSym [36], highlight the growing importance of comprehensive binary datasets. LLM4Decompile introduced a large LLM specifically trained for decompilation tasks, while ReSym combined LLMs with program analysis to recover both variable names and types. These works primarily focused on Linux binaries or specific aspects of decompilation. In contrast, our dataset provides a broader temporal perspective for the Windows platform, offering a rich foundation for training and evaluating models to exploit the patterns found in evolving projects. In CLAP [35], (Contrastive Language-Assembly Pre-training), the authors present a synthetic method of generating a dataset of binary code and corresponding natural language examples using LLMs. Our work complements theirs by utilizing real world projects that consist of thousands of lines of code and to a degree capture the evolution of a codebase over time. Training or fine-tuning an LLM on our dataset will potentially make generating synthetic large projects more representative of a typical real world open-source project.

While previous binary corpora have focused on breadth across different programs and configurations, we make the case that multiple versions represent a crucial dimension of diversity. This temporal depth enables researchers to study how binary artifacts evolve over time, providing invaluable training data for machine learning models tasked with binary analysis and reverse engineering tasks.

3 METHODS

We begin by describing how we selected projects to include in our manually-assembled deep history dataset, and detail study security-relevance by presenting some figures on CVEs present across versions. We next describe our modified build pipeline, before concluding with discussing our dataset per-se.

3.1 Project selection

We specifically targeted C and C++ projects that serve as foundational libraries or dependencies in larger software ecosystems, as these tend to have well-documented version histories and are consistently maintained. We manually identified potentially-relevant

Table 2: Projects with high severity CVEs

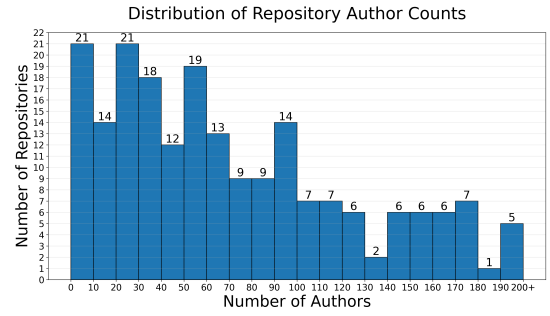
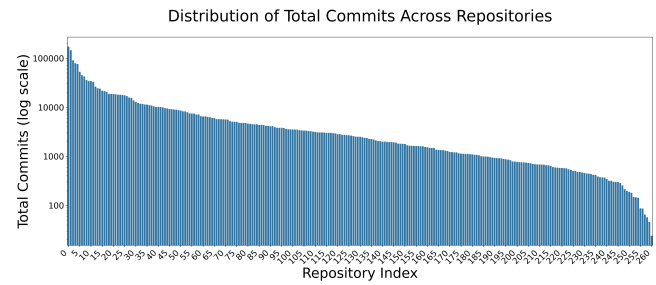
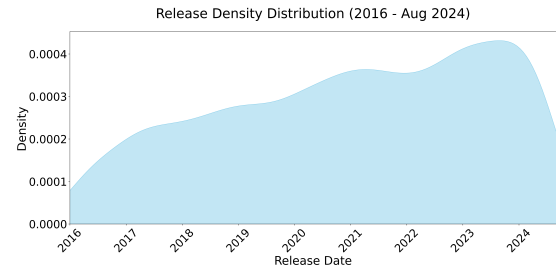
Projects	High severity CVEs
ffmpeg	159
openssl	79
freetype	59
h3	31
libxml2	28
libtiff	27
lua	21
libpng	17
wt	13
wolfssl	10

projects using a variety of methods (including input from collaborators, GitHub popularity, and other factors), focusing on projects that were not present in the original ASSEMBLAGE dataset. We focused on projects present on GitHub, using GitHub’s API to identify version and tag histories. We then divided projects into different categories based on their function and utility. The majority of them were related to tools for cryptography, libraries for manipulating different file formats, databases, image format processing, graphics libraries, networking and API clients, machine learning and mathematical libraries, libraries related to hardware and OS APIs. Table 1 details the breakdown of projects in various categories.

When analysing the build systems that the projects use, almost all of them supported the use of CMake [20]. Meson [33], a newer build system was favored by the newer projects. Certain projects employed their own methods of downloading and caching dependencies that had to be addressed by writing custom build scripts. As a proxy for popularity, we scanned the MITRE database for published CVEs for each of the projects. Some of the more popular ones had a large number of CVEs, but the vast majority of them had few if any. Due to the transitive dependency of software libraries on each other, the presence of vulnerabilities in the more popular base libraries has a magnified effect. As another measure of popularity we looked for the number of unique authors that contributed to the repository of each project. The number of authors ranged from 5 to more than 200, a majority of the projects had 10–100 authors.

3.2 Security-Relevance and Commit History

To study the relevance of the projects to our goals, we study two primary metrics: security-relevance and commit history. First, we analyzed the MITRE [27] Common Vulnerability and Exposures (CVE) database for security vulnerabilities associated with each project. As shown in Table 2, several projects had a significant number of CVEs, especially FFmpeg (159 high-severity CVEs) and OpenSSL (79 high-severity CVEs). Figure 3 presents the temporal distribution of releases from 2016 to 2024, showing an increasing trend in release frequency. This pattern suggests growing maturity in development practices across our dataset, with a notable acceleration in release density from 2020 onward. The distribution of author counts, as illustrated in Figure 1, indicates that the majority of projects main-tain development teams of 10-100 contributors, providing a rich diversity of programming styles and development methodologies.

**Figure 1: Distribution: the number of repositories having x number of authors****Figure 2: Distribution of the number of commits to each project; y-axis on log scale.****Figure 3: Kernel density estimate of GitHub release distributions across projects in our dataset.**

3.3 Build Pipeline

The first author spent roughly three months building the deep history binary corpus, averaging roughly a repository per day (plus the 151 repositories from Conan); this work included identifying relevant projects, manually writing build scripts (with the assistance of Claude upon occasion), and running the compilation across various versions. ASSEMBLAGE is written in Python, and includes an extensible API for interposing arbitrarily complex logic at many stages of the build pipeline. However, as mentioned, the current ASSEMBLAGE builders work in a mostly-automated fashion, manually parsing and modifying project configurations and using ad-hoc heuristics to handle potential build failures. By contrast, we wanted to target projects which required more complex build configurations (e.g.,

calling Python to generate a Makefile) that could not be anticipated by ASSEMBLAGE’s automated build tooling.

We first extended ASSEMBLAGE with new APIs to enable manually-specifying build configurations via a scriptable interface. Although we considered reusing an open-source package manager, we ultimately decided to implement our own API to take advantage of ASSEMBLAGE’s standardized data formats and robust tooling. Our new “custom builder” APIs are readily compatible with standard package managers: as evidence of this, we also implemented a Conan-based builder; Conan is a more recent Windows package manager which includes multiple versions of each repository.

Listing 1 illustrates an example custom build scripts written using our new API. The script utilizes our new CustomWindowsBuild class to manage the build process across multiple versions of Microsoft’s C++ REST SDK, a modern library for cloud-based client-server communication. The hook takes various parameters (such as architecture and optimization level), passed in from the higher-level ASSEMBLAGE distributed build architecture. In constructing the builder, we specify certain key properties such as the GitHub URL, the build mode we desire (set here to one mode—though nothing prevents diversifying across modes in principle), and a list of version tags (here v2.10.16 to v2.10.19).

The build process is customized through two primary hooks. The first is `install_prerequisites_hook`, which manages dependency installation. In this case (line 4) we use `vcpkg`, Microsoft’s C++ package manager, to install dependencies; some projects (e.g., Boost and Qt) included bundled build scripts that required we invoke them *after* configuring the required environment variables. In our `install_prerequisites_hook`, we use `vcpkg` to install `zlib`, `OpenSSL`, and various Boost components (`system`, `date-time`, `regex`, and `interprocess`), along with `websocket` libraries. Next, `build_hook` implements a CMake-based build pipeline that maintains consistency across different architectures; this project was designed to use CMake, and hence failed to build using ASSEMBLAGE’s default build tooling. In this case, we use Visual Studio 2022’s build tools and integrate with `vcpkg` through its toolchain file, ensuring reproducible builds across different versions. The build process is configured to handle multiple architectures through an architecture mapping system that translates between Visual Studio’s `vcvarsall` conventions and CMake’s architecture specifications.

In this case, our script targets four releases of `CppRestSDK` (v2.10.16 through v2.10.19). This approach allows us to study how changes in the library’s implementation affect its binary footprint while maintaining API compatibility. The build configuration maintains separate build directories for each architecture, preventing cross-contamination between different builds and ensuring clean compilation for each version. To make our corpus useful for reverse engineering, we built each project using the `Debug` and `RelWithDebugInfo` compiler options, which generate a PDB file containing source line information. The generated debug symbol databases for each binary allowed us to create a mapping from portions of the binary (at the granularity of functions) to lines of source code. After the compilation is finished, the regular post-processing of the generated binaries and PDB files occurs.

Approximately 30% of these build scripts were initially generated using Anthropic’s Claude [2] model. We prompted it with instructions from the README file and what structure the resulting script

```
def install_prerequisites_hook():
    print("Installing prerequisites...")
    # Install dependencies using vcpkg
    vcpkg_cmd = ("vcpkg install --triplet x64-windows zlib openssl
    <-- boost-system " +
                "boost-date-time boost-regex boost-interprocess
    <-- websocketpp brotli")
    cmd_with_output(vcpkg_cmd, platform="windows")
    print("Done")

def build_hook(build_dir,
               clone_dir,
               build_mode,
               arch,
               optimization):
    logging.info("Building CppRestSDK...")
    vcvarsall_loc = "\\C:\\\\Program Files\\Microsoft Visual
    <-- Studio\\2022\\Community\\VC\\Auxiliary\\Build\\vcvarsall.bat\\"
    vcpkg_toolchain = "C:\\\\vcpkg\\scripts\\buildsystems\\vcpkg.cmake"
    # Maps vcvars arch to cmake preset arch
    arch_map = {'x64': 'x64', 'x64_arm': 'ARM', 'x64_arm64': 'ARM64',
    <-- 'x86': 'x86'}
    build_cmd = (
        f"vcvarsall_loc {arch} && " +
        f"cd Release && mkdir build.{arch} && cd build.{arch} && " +
        f"cmake .. -A {arch_map[arch]} " +
        f"-DCMAKE_TOOLCHAIN_FILE={vcpkg_toolchain} " +
        f"-DCMAKE_BUILD_TYPE={build_mode} && " +
        f"cmake --build . --config {build_mode}"
    )
    obj = CustomWindowsBuild(
        clone_dir='C:\\\\CppRestSDK\\',
        clone_flags='',
        collect_dir='C:\\\\Binaries',
        source_dir='C:\\\\CppRestSDK\\',
        build_dir=f'C:\\\\CppRestSDK\\Release\\build.{arch}\\',
        project_git_url='https://github.com/Microsoft/cpprestsdk.git',
        optimization='',
        build_mode='Release', # Default to Release build
        arch=arch,
        tags=['v2.10.19', 'v2.10.18', 'v2.10.17', 'v2.10.16'], # Recent
    <-- release tags
        build_hook=build_hook,
        install_prerequisites_hook=install_prerequisites_hook,
        compiler_version="Visual Studio 17 2022"
    )
    obj.run()
```

Listing 1: cpprestsdk Build Configuration

must adhere to. We then manually refined the generated scripts to ensure reliability and compatibility with our build environment. An AWS server (32 GB of RAM and 6 CPU cores) was used to perform the compilation. The builds were run concurrently, typically 6 builds simultaneously. We used the existing Assemblage infrastructure to perform the post-processing of the compiled binaries. This process extracted function names from each binary. It then created a database that mapped each function to its relative location (RVA) within the binary. The source line information was used to map the function to the original source code file (and line number within it). We used the native Windows PDB SDK to obtain debugging information from the generated PDB files.

3.4 Dataset

The dataset consists of a SQLite database and a compressed zip file. The SQLite database holds metadata about the binaries, functions within them, and the mapping from source lines to chunks of the

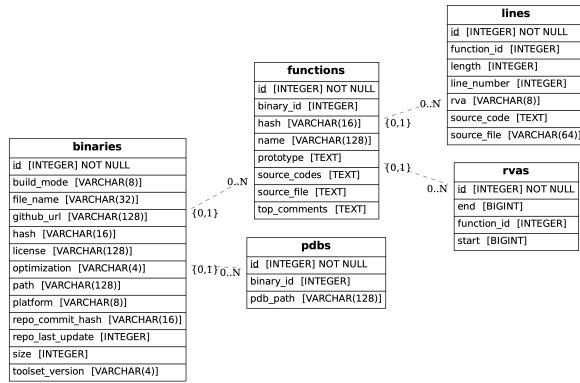


Figure 4: Entity Relationship diagram displaying the schema of the Sqlite database.

binaries. The binaries and corresponding pdb files are compressed into a zip file which accompanies the sqlite database.

The metadata includes the following tables, with the schema as described by Figure 4:

- **Binaries:** Each binary has an entry in this table. Information about the project’s GitHub repository, commit hash, compiler used, compilation options, and platform are provided.
- **Functions:** Each function has an entry in this table. There is a Many to One mapping from functions back to its originating binary. Fields include the function hash, original source line, extracted comments, and prototype.
- **Lines:** Contains detailed source code information at the line level, maintaining a many-to-one relationship with functions. Each entry includes the line number, length, relative virtual address (RVA), and the actual source code content with its file location.
- **PDBs:** Stores Program Database (PDB) file information associated with binaries, maintaining a many-to-one relationship with the binaries table. Each entry contains the path to the PDB file and links back to its binary through `binary_id`.
- **RVAs:** Manages relative virtual address ranges for functions, storing start and end addresses. This table maintains a many-to-one relationship with functions through the `function_id` field.

4 EVALUATION AND RESULTS

We conducted several evaluations of our deep history dataset to illustrate its potential relevance to knowledge discovery and information retrieval over binary corpuses. First, we processed the raw binaries through MalConv, extracting 256-dimensional semantic embeddings that preserved the functional characteristics of each executable without relying on disassembly or decompilation. The dataset’s version lineage properties were analyzed through comparing the embeddings across versions, in order to reveal consistency in semantic properties of the binaries across versions. We then

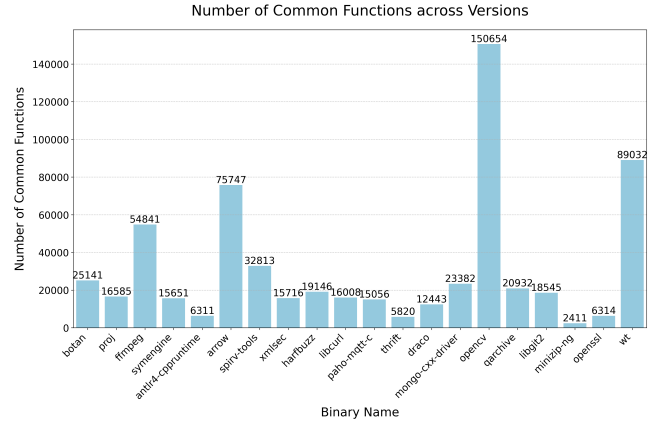


Figure 5: A bar plot of the number of functions with the same name across versions for a select sample of binaries

evaluated the dataset’s utility for information retrieval tasks by implementing an approximate nearest neighbor search using FAISS, optimized with parameters ($k=30$, $M=30$) that balanced precision and computational efficiency. Finally, we employed Linear Bayesian Modeling to investigate the relationship between code evolution factors and binary similarity, constructing a model with four key features: temporal distance, file changes, feature distance, and commit frequency.

4.1 MalConv Feature Analysis for Binary Similarity

To quantify the semantic relationships and temporal evolution across different versions of identical binary software, we conducted a comprehensive feature analysis utilizing deep learning techniques. This section presents our methodology and findings regarding the stability and differentiation of binary features across software versions.

4.1.1 Feature Extraction Methodology. Our feature extraction process employed MalConv [29], a specialized Convolutional Neural Network architecture originally designed for malware classification and trained on the extensive Assemblage dataset of Windows executables. The architecture was selected for its demonstrated capability to capture semantic properties of binary code without requiring disassembly or decompilation. The MalConv model processes binaries through the following pipeline:

- (1) **Byte Embedding Layer:** Each raw byte from the binary file is transformed into a learned 8-dimensional feature vector. This transformation enables the model to differentiate between numerically adjacent bytes that may represent entirely different semantic constructs in the binary context. For instance, the byte values `0x74` and `0x75` (ASCII ‘t’ and ‘u’) have minimal numeric differences but potentially significant semantic distinctions in executable code. Copy
- (2) **Convolutional Processing:** The embedded byte sequences are processed through temporal 1-D convolutional layers with a kernel size of 512 and 128 filters. These layers identify patterns across varying contextual windows within the

binary structure, capturing both local instruction patterns and broader structural features.

- (3) **Global Feature Aggregation:** A temporal max-pooling operation is applied across the convolutional output to extract the most salient features.
- (4) **Dimensional Reduction:** A fully-connected layer with 256 neurons maps the pooled features to a dense embedding space, producing a fixed-length vector representation of each binary that encapsulates its functional characteristics.

The resulting 256-dimensional embedding vectors serve as compact representations of the binaries' semantic content. To quantify similarity between these high-dimensional embeddings, we employed cosine similarity as our primary metric, defined as:

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

where A and B represent the embedding vectors of two binary files. This metric was selected for its effectiveness in comparing directional relationships in high-dimensional spaces while normalizing for magnitude differences.

4.1.2 Cross-Version Stability Analysis. Our analysis of feature stability across different versions of the same software package revealed remarkable consistency, as visualized in Figure 7. The minimum cosine similarity across all package versions maintained a threshold of 0.697, with the majority of packages exhibiting minimum similarities above 0.80. This finding is particularly significant as it demonstrates that substantial code modifications between versions do not fundamentally alter the semantic signature captured by our embedding approach. Statistical analysis of the similarity distribution yielded the following key metrics:

- Mean cross-version similarity: 0.9447 ($\sigma = 0.068$)
- Median cross-version similarity: 0.9587
- Minimum observed similarity: 0.697
- Maximum observed similarity: 0.999

The high mean similarity value (0.94) with a relatively low standard deviation indicates strong consistency in the semantic representations across versions. Even the minimum observed similarity (0.697) remains substantially above what would be expected for unrelated binaries, suggesting that core functional characteristics persist throughout a software's evolution cycle.

4.1.3 Pairwise Similarity Patterns. Figure 6 presents a heatmap visualization of pairwise cosine similarities between binary embeddings across our dataset. The visualization reveals several patterns:

Diagonal Clusters: The prominent diagonal blocks of high similarity (dark blue) correspond to different versions of the same software package. These clusters demonstrate that version lineage is captured in the embedding space, with sequential versions exhibiting high similarity.

Off-Diagonal Relationships: Areas of moderate similarity (medium blue, approximately 0.85-0.90) outside the diagonal blocks indicate potential relationships between different software packages. We hypothesize they potentially represent:

- Shared dependencies or third-party libraries

- Common programming patterns specific to certain application domains
- Code reuse or forking relationships between projects

Distinct Software Families: The clear delineation between certain clusters (lighter regions between dark blocks) demonstrates the model's ability to discriminate between fundamentally different software functionalities.

Gradient Patterns: Within some diagonal clusters, a gradual decrease in similarity can be observed as the version distance increases, illustrating the cumulative effect of incremental changes over multiple version transitions. The cross-package similarity baseline typically ranges between 0.75-0.85, significantly lower than the intra-package similarity of 0.94 on average. This differential confirms the embedding space effectively captures both the commonalities within software evolution and the distinctions between disparate software packages.

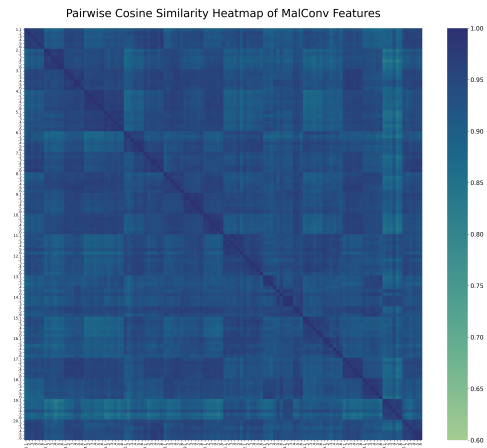


Figure 6: Heat map visualization of pairwise cosine similarities between generated embeddings. Darker colors indicate higher similarity between binary pairs, revealing potential clusters of related functionality or shared code.

4.2 Category-wise Nearest Neighbors

A fundamental challenge in reverse engineering is the identification of similar binary patterns or function signatures across various software artifacts [8]. To address this challenge, we partitioned our binary dataset into 15 distinct software categories, encompassing various applications and their respective versions. This categorization facilitates the simulation of real-world scenarios where analysts seek to identify similar functional components within binary files to examine corresponding source code segments. For feature extraction, we reused the embeddings generated by MalConv. To efficiently query and retrieve similar binaries, we implemented an approximate nearest neighbor (ANN) [3] search using Meta's FAISS library [7]. The Nearest Neighbors (kNN) index was constructed with the following parameters:

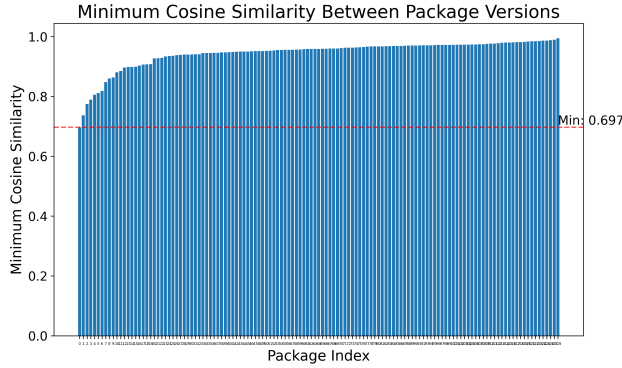


Figure 7: A bar plot of the minimum cosine similarities across versions for 20 binaries

- $k = 30$ (number of neighbors to retrieve)
- $M = 30$ (number of connections per layer in the index graph)
- Distance metric = cosine similarity

The performance of our binary similarity detection approach was evaluated using standard information retrieval metrics, as presented in Table 3. The mean reciprocal rank (MRR) metric, which quantifies the effectiveness of the retrieval system by measuring the inverse of the rank at which the first relevant document is retrieved, ranged between 0.8612 and 0.8616 with 95% confidence. This indicates a decently high precision in identifying the most relevant similar binaries.

Metric	Value
MRR Lower Bound	0.8612
MRR Upper Bound	0.8616
Recall@1	0.8020
Recall@2	0.8672
Recall@5	0.9424
Recall@10	0.9687
Evaluation Time (s)	0.0096

Table 3: Performance Metrics of running k -Nearest Neighbors ($K=30$) on our dataset

The Recall@ k metrics provide further insight into the retrieval performance. Recall@1 of 0.8020 demonstrates that our approach correctly identifies the most similar binary on the first attempt in 80.20% of cases. This performance improves significantly when considering a broader set of candidates, with Recall@5 reaching 0.9424, indicating that the correct match appears within the top 5 retrieved results in 94.24% of cases. Figure 8 presents the normalized confusion matrix derived from our kNN classification results. The matrix reveals several noteworthy patterns of similarity between related software categories that merit further discussion.

The diagonal elements of the matrix represent the proportion of correctly classified instances within each category, with higher values indicating better classification performance. Categories such as machine learning libraries (0.86), formatting utilities (0.83), and

mathematical libraries (0.83) demonstrate particularly strong intra-category cohesion, suggesting distinct binary signatures that are readily distinguishable from other categories.

Analysis of off-diagonal elements provides valuable insights into inter-category relationships. For instance:

- Graphics libraries are occasionally misclassified as image processing software (0.11), and vice versa (0.05), reflecting their shared functionality in rendering and manipulating visual data.
- Cryptography libraries show minor confusion with networking implementations (0.10), likely due to their common inclusion of secure communication protocols and data encryption routines.
- Data structures libraries exhibit some misclassification as database systems (0.02) and parsing utilities (0.07), indicating overlapping patterns in data organization and manipulation algorithms.
- System tools demonstrate confusion with operating system APIs (0.05), reflecting their shared low-level system interaction characteristics.
- Compression algorithms show notable similarity to file format processors (0.07), highlighting their common functionality in data transformation and encoding.

These observed patterns of similarity align with theoretical expectations based on functional relationships between software categories. Analysts should anticipate potential false positives between categories with higher confusion rates, such as system tools and operating system APIs. Conversely, the high discrimination between categories like machine learning libraries and compression utilities suggests that matches between these domains are likely to be genuine.

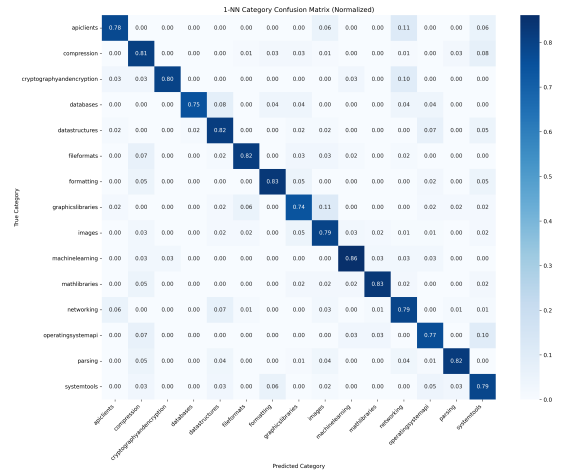


Figure 8: A confusion matrix of categories based on running a 1-NN

4.3 Linear Bayesian Model

The Deep Histories dataset also allows for investigation of the relationship between code evolution and binary similarity, which

can be critical for maintaining databases of known functions that are used for both being and malicious purposes (e.g., encryption). We perform a linear Bayesian analysis to demonstrate the avenue of research this enables. Our model is computed using Markov Chain Monte Carlo [24] using the NUTS sampler [16] and implemented with Numpyro [28]. The $\hat{R}$ scores for all variables is ≤ 1.005 , indicating convergence [12]. All distributions were Gaussian or half-Gaussian (positive values only) as appropriate, and the target variable was modeled as a Beta regression as $\text{Beta}(\theta|\mu, \kappa) = \frac{1}{B(\mu\kappa, (1-\mu)\kappa)} \theta^{\mu\kappa-1} (1-\theta)^{(1-\mu)\kappa-1}$ what priors $\kappa \in \mathbb{R}^+$ and mean estimate $\theta \in (0, 1)$ and target value $\in (0, 1)$.

Our Linear Bayesian Model analysis sought to understand the factors that influence binary similarity across versions. Each feature of interest is modeled with a hyper-prior (the global rate), which is used as the prior for a project-specific version of the covariate. In this way, the Bayesian approach shares information across all projects while also recognizing that the multiple compilations of one project have an intra-source similarity that should not influence other projects [11].

We constructed the model using four key features:

- The number of days between each pair of releases
- The number of source files changed between each pair of releases normalized by total number of files
- The cosine distance between the observed MalConv features between each pair of releases
- The number of commits between each pair of releases.

To determine that such a model is reasonable to draw any inferences from, a posterior predictive check is used to compare the predictions with observed values from the model [10]. This is performed in Figure 9 and shows that our model successfully captures the general distribution of similarity scores, though it struggles to fully model the extreme cases where similarity approaches 1.0. This limitation suggests the presence of non-linear relationships that our linear model cannot capture.

Resolving a model that “fully captures” the distribution, or if such a model exists, is beyond the scope of this article — which introduces the dataset and demonstrates the potential research value. Still, under our linear model, we can examine the hyper-prior’s Highest Density Interval (HDI, similar to a confidence interval) to determine if a statistically significant relationship exists between our covariates and outcome (file similarity). These HDIs are shown in Figure 10 confirmed our expectation that temporal distance correlates with reduced similarity; it revealed that the raw number of changed files had less impact than expected. The hyper-prior analysis showed the strongest effects from the bias term and file change presentation features, with coefficients of 2.3 and 1.7, respectively.

The results from Figure 10 are again the hyper-prior measuring global rates. A Bayesian model allows us to simultaneously model the per-project local rates. The distribution of each variable’s per-project value as it differs from the global value is shown as a kernel density estimate in Figure 11, showing that most projects are similar to the global prior — a necessity for the linear model to be a reasonable choice as shown by our posterior predictive check.

However, our kernel density estimation of project-specific coefficients revealed approximately 15% demonstrate significantly different behavior, forming a distinct tail in the distribution past 1.0

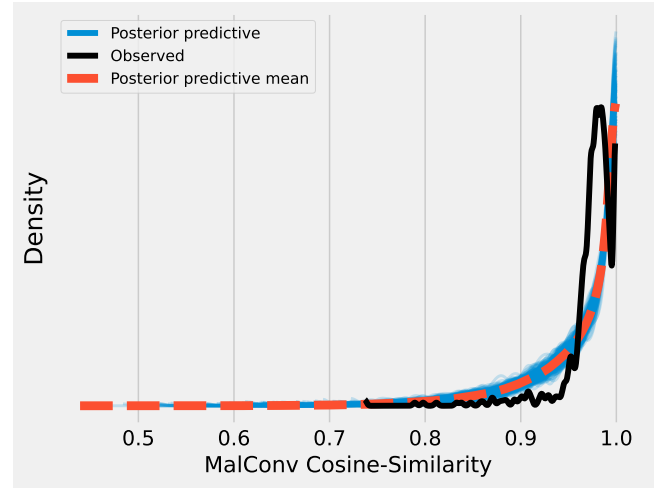


Figure 9: Posterior predictive check of the linear Bayesian model vs the observed MalConv similarity scores. The scores are reasonably predictable but can not capture a non-linearity in the output distribution.

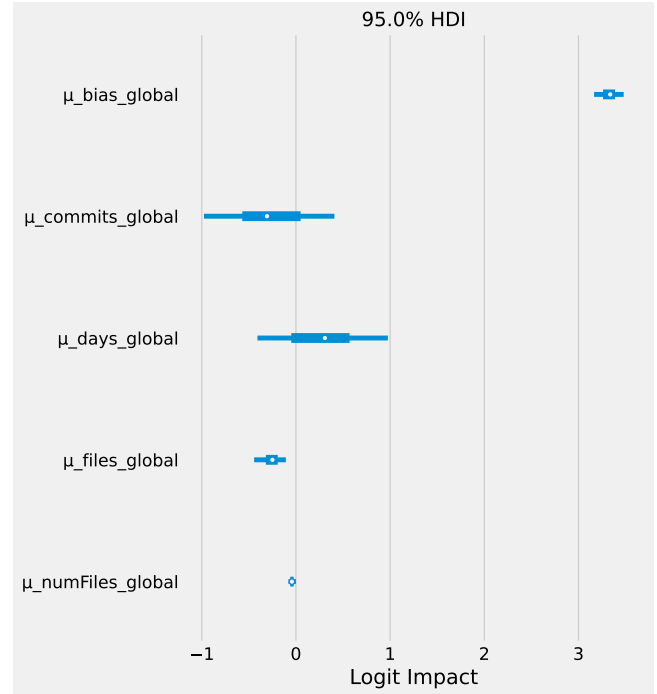


Figure 10: Hyper-Priors for the bias term and four variables show that the bias and presentation of file changes are the most impactful. However, changing more files counter-intuitively reduces the predicted similarity.

standardized units away from the mean. This shows that there are projects that are “contrarian” in the impact that each variable has on the (linear) model of file similarity. This shows both a potential

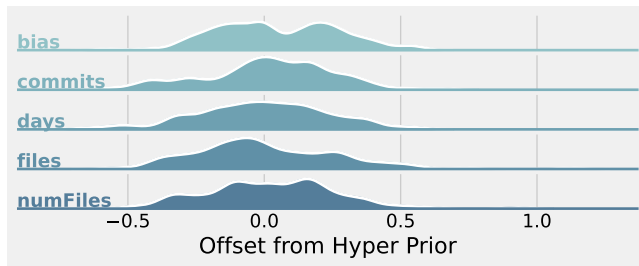


Figure 11: KDE of the offset of each project-specific coefficient from the hyper-prior. Most projects are the same, but the tail of the distribution near 1 indicates some “contrarian” projects with different behavior from the norm.

for using Bayesian modeling to identify contrarian projects for separate investigation (e.g., could explain why a project is not seen in wide use is because it is not easily recognized).

5 CONCLUSION

Research in code-binary mappings, malware analysis, disassembly, and code-LLMs, are among the many areas hindered by a lack of higher-quality data. We introduce the Assemblage Deep-Histories dataset, which contains code-binary mappings for 251 major real-world projects with multi-year histories. By developing custom scripts to compile across those histories with varying compilers, we enable a wide breadth of potential research that is not possible with existing corpora. This was demonstrated with an examination of existing tool’s performance in function similarity, functional classification, and a Bayesian analysis of impacts in function similarity.

REFERENCES

- [1] H. Anderson and Phil Roth. 2018. EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models. *ArXiv abs/1804.04637* (2018). <https://api.semanticscholar.org/CorpusID:4888440>
- [2] Anthropic. 2024. *Claude 3 Model Card*. Technical Report. Anthropic. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c3b80b7bbc618857627/Model_Card_Claude_3.pdf Accessed: 2025-02-24.
- [3] Sunil Arya, David M. Mount, Nathan S. Netanyahu, Ruth Silverman, and Angela Y. Wu. 1998. An optimal algorithm for approximate nearest neighbor searching fixed dimensions. *J. ACM* 45 (1998), 891–923. <https://api.semanticscholar.org/CorpusID:8193729>
- [4] Ferhat Özgür Çatak and Ahmet Faruk Yazı. 2019. A Benchmark API Call Dataset for Windows PE Malware Classification. *CoRR abs/1905.01999* (2019). <http://arxiv.org/abs/1905.01999>
- [5] Francesca Console, Giuseppe D’Aquanno, Giuseppe Antonio Di Luna, and Leonardo Querzoni. 2023. BinBench: a benchmark for x64 portable operating system interface binary function representations. *PeerJ Computer Science* 9 (2023). <https://api.semanticscholar.org/CorpusID:259029804>
- [6] Dragos Georgian Corlatescu, Alexandru Dinu, Mihaela Gaman, and Paul Sume-drea. 2023. EMBERSim: A Large-Scale Databank for Boosting Similarity Search in Malware Analysis. *ArXiv abs/2310.01835* (2023). <https://api.semanticscholar.org/CorpusID:263608542>
- [7] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). [arXiv:2401.08281 \[cs.LG\]](https://arxiv.org/abs/2401.08281)
- [8] Jiang Du, Qiang Wei, Yisen Wang, and Xiangjie Sun. 2023. A Review of Deep Learning-Based Binary Code Similarity Analysis. *Electronics* (2023). <https://api.semanticscholar.org/CorpusID:265267109>
- [9] Weike Fang, Zhejian Zhou, Junzhou He, and Weihang Wang. 2024. StackSight: Unveiling WebAssembly through Large Language Models and Neurosymbolic Chain-of-Thought Decompilation. *ArXiv abs/2406.04568* (2024). <https://api.semanticscholar.org/CorpusID:270357304>
- [10] Andrew Gelman. 2006. Prior distributions for variance parameters in hierarchical models (comment on article by Browne and Draper). *Bayesian Analysis* 1, 3 (2006), 515 – 534. <https://doi.org/10.1214/06-BA117A>
- [11] Andrew Gelman, Jennifer Hill, and Masanao Yajima. 2012. Why We (Usually) Don’t Have to Worry About Multiple Comparisons. *Journal of Research on Educational Effectiveness* 5, 2 (April 2012), 189–211. <https://doi.org/10.1080/19345747.2011.618213>
- [12] Andrew Gelman and Donald B. Rubin. 1992. Inference from Iterative Simulation Using Multiple Sequences. *Statist. Sci.* 7, 4 (1992), 457–472. <http://www.jstor.org/stable/2246093>
- [13] Harrison Green, Edward J. Schwartz, Claire Le Goues, and Bogdan Vasilescu. 2024. STRIDE: Simple Type Recognition In Decompiled Executables. *arXiv:2407.02733 [cs.CR]* <https://arxiv.org/abs/2407.02733>
- [14] Richard E. Harang and Ethan M. Rudd. 2020. SOREL-20M: A Large Scale Benchmark Dataset for Malicious PE Detection. *ArXiv abs/2012.07634* (2020). <https://api.semanticscholar.org/CorpusID:229153216>
- [15] Xu He, Shu Wang, Yunlong Xing, Pengbin Feng, Haining Wang, Qi Li, Songqing Chen, and Kun Sun. 2022. BinProv: Binary Code Provenance Identification without Disassembly. *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses* (2022). <https://api.semanticscholar.org/CorpusID:252910574>
- [16] M.D. Hoffman and A. Gelman. 2014. The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo. *J Mach Learn Res* 15 (2014), 1593–1623. <http://mcmc-jags.sourceforge.net>. Accessed September 12, 2021..
- [17] JFrog. [n. d.]. *Conan: C/C++ Package Manager*. JFrog Ltd. <https://conan.io/>
- [18] Robert J. Joyce, Booz Allen Hamilton, Dev Amlani, and Edward Raff. 2021. MOTIF: A Large Malware Reference Dataset with Ground Truth Family Labels. *ArXiv abs/2111.15031* (2021). <https://api.semanticscholar.org/CorpusID:237425806>
- [19] Dongkwan Kim, Eunsoo Kim, Sang Kil Cha, Soeul Son, and Yongdae Kim. 2022. Revisiting Binary Code Similarity Analysis using Interpretable Feature Engineering and Lessons Learned. *IEEE Transactions on Software Engineering* (2022), 1–23. <https://doi.org/10.1109/TSE.2022.3187689>
- [20] Kitware, Inc. 2024. CMake. <https://cmake.org/>
- [21] Xuezixiang Li, Qu Yu, and Heng Yin. 2021. PalmTree: Learning an Assembly Language Model for Instruction Embedding. *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (2021). <https://api.semanticscholar.org/CorpusID:232134887>
- [22] Chang Liu, Rebecca Saul, Yihao Sun, Edward Raff, Maya Fuchs, Townsend Southard Pantano, James Holt, and Kristopher Micinski. 2024. Assemblage: Automatic Binary Dataset Construction for Machine Learning. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 58698–58715. https://proceedings.neurips.cc/paper_files/paper/2024/file/6bbefc73a187dd42e0dc065b4e7a0615-Paper-Datasets_and_Benchmarks_Track.pdf
- [23] Zhenhao Luo, Pengfei Wang, Baosheng Wang, Yong Tang, Wei Xie, Xu Zhou, Danjun Liu, and Kai Lu. 2023. VulHawk: Cross-architecture Vulnerability Detection with Entropy-based Binary Code Search. *Proceedings 2023 Network and Distributed System Security Symposium* (2023). <https://api.semanticscholar.org/CorpusID:257501992>
- [24] N. Metropolis, A.W. Rosenbluth, M.N. Rosenbluth, A.H. Teller, and E. Teller. 1953. Equation of state calculations by fast computing machines. *J Chem Phys* 21, 6 (1953), 1087–1092. <https://doi.org/10.1063/1.1699114>
- [25] Microsoft. [n. d.]. *MSBuild*. Microsoft Corporation. <https://learn.microsoft.com/en-us/visualstudio/msbuild/msbuild> Accessed: YYYY-MM-DD.
- [26] Microsoft. [n. d.]. *vcpkg*. Microsoft Corporation. <https://github.com/microsoft/vcpkg>
- [27] MITRE Corporation. [n. d.]. Common Vulnerabilities and Exposures (CVE). <https://cve.mitre.org/>
- [28] D. Phan, N. Pradhan, and M. Jankowiak. [n. d.]. Composable Effects for Flexible and Accelerated Probabilistic Programming in NumPyro. December 2019. <http://arxiv.org/abs/1912.11554>. Accessed September 12, 2021..
- [29] Edward Raff, William Fleshman, Richard Zak, Hyrum Anderson, Bobby Filar, and Mark Mclean. 2021. Classifying Sequences of Extreme Length with Constant Memory Applied to Malware Detection. In *The Thirty-Fifth AAAI Conference on Artificial Intelligence*. <https://arxiv.org/abs/2012.09390>
- [30] Rebecca Saul, Chang Liu, Noah Fleischmann, Richard Zak, Kristopher Micinski, Edward Raff, and James Holt. 2024. Is Function Similarity Over-Engineered? Building a Benchmark. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 21636–21655. https://proceedings.neurips.cc/paper_files/paper/2024/file/2663c994c84a79b338bca613fe1ae223-Paper-Datasets_and_Benchmarks_Track.pdf
- [31] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Miao Shi, and Yang Liu. 2024. LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs’ Vulnerability Reasoning. *ArXiv abs/2401.16185* (2024). <https://api.semanticscholar.org/CorpusID:267311934>
- [32] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. LLM4Decompile: Decompiling Binary Code with Large Language Models. *arXiv:2403.05286 [cs.PL]*

- [33] The Meson Development Team. 2024. Meson Build System. <https://mesonbuild.com/>
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [35] Hao Wang, Zeyu Gao, Chao Zhang, Zihan Sha, Mingyang Sun, Yuchen Zhou, Wenyu Zhu, Wenju Sun, Han Qiu, and Xi Xiao. 2024. CLAP: Learning Transferable Binary Code Representations with Natural Language Supervision. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 503–515. <https://doi.org/10.1145/3650212.3652145>
- [36] Danning Xie, Zhuo Zhang, Nan Jiang, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. 2024. ReSym: Harnessing LLMs to Recover Variable and Data Structure Symbols from Stripped Binaries. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) (CCS '24). Association for Computing Machinery, New York, NY, USA, 4554–4568. <https://doi.org/10.1145/3658644.3670340>
- [37] Muhammad Irfan Yousuf, Izza Anwer, Tanzeela Shakir, Minahil Siddiqui, and Maysoon Shahid. 2022. Multi-feature Dataset for Windows PE Malware Classification. *ArXiv abs/2210.16285* (2022). <https://api.semanticscholar.org/CorpusID:253224277>
- [38] Yun Zhang, Yuling Liu, Ge Cheng, and Bo Ou. 2024. GTrans: Graph Transformer-Based Obfuscation-resilient Binary Code Similarity Detection. *Proceedings 2024 Workshop on Binary Analysis Research* (2024). <https://api.semanticscholar.org/CorpusID:270931913>
- [39] Fei Zuo, Cody Tompkins, Qiang Zeng, Lannan Luo, Yung Ryn Choe, and Junghwan Rhee. 2024. BinSimDB: Benchmark Dataset Construction for Fine-Grained Binary Code Similarity Analysis. *ArXiv abs/2410.10163* (2024). <https://api.semanticscholar.org/CorpusID:273346788>

A APPENDIX

A.1 List of projects

The list of projects are presented in the following table.

Table 4: List of projects

sqlite3	yyjson	freetype	json-c	libpq
imgui	7bitdi	ftxui	lerc	libsodium
catch2	blend2d	gdc	libaom-av1	libtins
flecs	cgltf	gtest	libavrocpp	libusb
fnt	draco	h3	libgd	libvips
re2	geos	highs	libmaxminddb	libx264
flatbuffers	harfbuzz	icu	libmysqlclient	libxslt
hdf5	libavif	implot	libsndfile	log4cxx
highway	libressl	libcbor	libsrtp	matio
libxml2	mongo-cxx-driver	libdwarf	libsvm	minizip
miniaudio	nghttp3	libftp	libsvtav1	minizip
openssl	nsync	libssh2	libtorrent	mpfr
poco	onnx	libtiff	libvpx	opencolorio
simdjson	opencl-icd-loader	libyuv	libwebp	openh264
wolfssl	pkgconf	libzip	libwebsockets	paho-mqtt-cpp
zlib-ng	proj	meshoptimizer	libzipp	prometheus-cpp
libcurl	spirv-cross	mimalloc	lightgbm	pugixml
libdeflate	wasmtime	nspr	log4cplus	qt
libgit2	ada	octomap	mailio	rmlui
libpqxx	botan	opencv	mariadb-connector-c	screen_capture_lite
minizip-ng	c-blosc2	openimageio	mongo-c-driver	sfml
openexr	c4core	plutovg	mozjpeg	subjectizer
c-blosc	cjson	rabbitmq-c	mppp	sqlite3mc
cppttrace	cpr	rapidyaml	msgpack-c	tgbot
ixwebsocket	cwalk	rotor	onetbb	tinycluster
kuba-zip	double-conversion	snappy	onnxruntime	tree-gen
libnghttp2	fast-cdr	spirv-tools	openal	vsg
libpng	g3log	tesseract	openjpeg	wasm-micro-runtime
libuv	jsoncpp	tree-sitter-c	qcbor	zeromq
lunasvg	libheif	uriparser	qr-code-generator	zstd
mosquitto	libjpeg-turbo	utf8proc	rocksdb	
nng	libxlsxwriter	wt	sail	
openblas	lua	xnnpack	sofa	
simdutf	pdf-writer	zimg	tinyspline	
spdlog	qarchive	zxing-cpp	xerces-c	
sqlcipher	soplex	aeron	zlib	
benchmark	symengine	angelscript	gdk-pixbuf	
boost	thrift	armadillo	glad	
bzip3	tracy	asmjit	gm2calc	
c-ares	xz_utils	cargs	graphene	
expat	z3	clipper2	hazelcast-cpp-client	
libassert	antlr4-cppruntime	cpprestsdk	hdrhistogram-c	
librdkafka	arrow	cpuinfo	hiredis	
paho-mqtt-c	assimp	cpython	json-schema-validator	
pcre2	bdwgc	date	leptonica	
redis-plus-plus	calceph	ezc3d	lexbor	
roaring	cfitsio	fruit	libarchive	